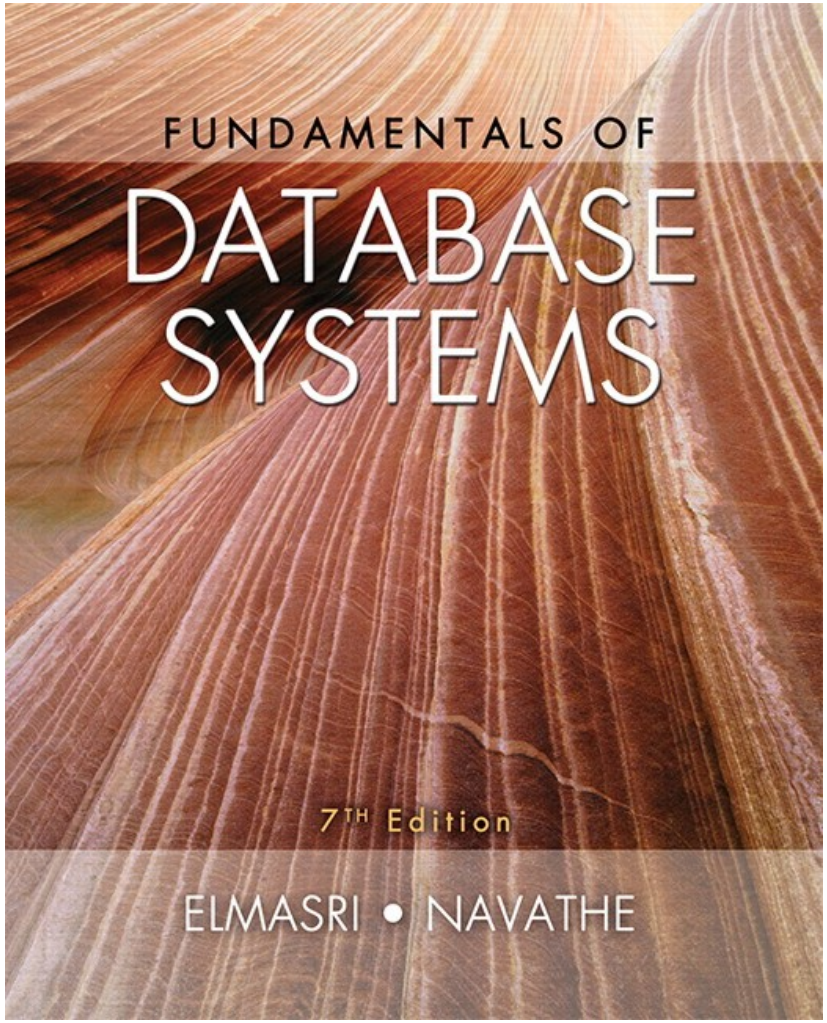


CHAPTER 14

Basics of Functional Dependencies and Normalization for Relational Databases

Chapter Outline

- 1 Informal Design Guidelines for Relational Databases
  - 1.1 Semantics of the Relation Attributes
  - 1.2 Redundant Information in Tuples and Update Anomalies
  - 1.3 Null Values in Tuples
  - 1.4 Spurious Tuples
- 2 Functional Dependencies (FDs)
  - 2.1 Definition of Functional Dependency

Chapter Outline

- 3 Normal Forms Based on Primary Keys
  - 3.1 Normalization of Relations
  - 3.2 Practical Use of Normal Forms
  - 3.3 Definitions of Keys and Attributes Participating in Keys
  - 3.4 First Normal Form
  - 3.5 Second Normal Form
  - 3.6 Third Normal Form
- 4 General Normal Form Definitions for 2NF and 3NF (For Multiple Candidate Keys)
- 5 BCNF (Boyce-Codd Normal Form)
- 6 Multivalued Dependency and Fourth Normal Form

1. Informal Design Guidelines for Relational Databases (1)

- What is relational database design?
  - The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
  - The logical "user view" level
  - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
  - - 1NF (First Normal Form)
  - - 2NF (Second Normal Form)
  - - 3NF (Third Normal Form)
  - - BCNF (Boyce-Codd Normal Form)
- Additional types of dependencies, further normal forms, relational design algorithms by synthesis are discussed in Chapter 15

1.1 Semantics of the Relational Attributes must be clear

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
  - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
  - Only foreign keys should be used to refer to other entities
  - Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

Figure 14.1 A simplified COMPANY relational database schema

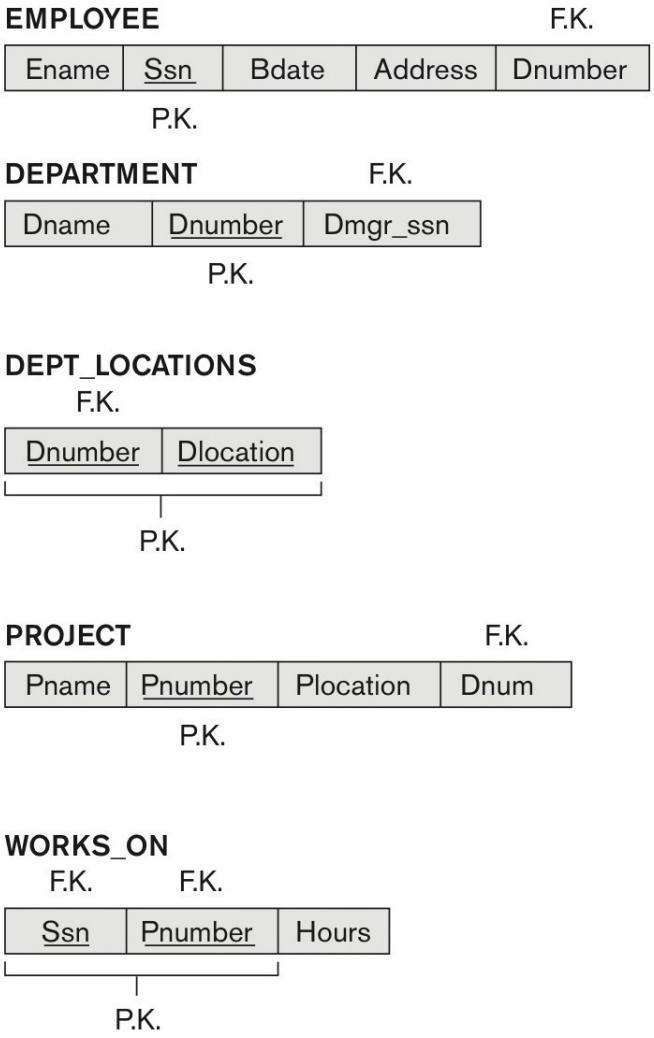


Figure 14.1 A simplified COMPANY relational database schema.

1.2 Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
  - Wastes storage
  - Causes problems with update anomalies
    - Insertion anomalies
    - Deletion anomalies
    - Modification anomalies

EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Update Anomaly:
  - Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
  - Cannot insert a project unless an employee is assigned to it.
- Conversely
  - Cannot insert an employee unless an he/she is assigned to a project.

EXAMPLE OF A DELETE ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
  - When a project is deleted, it will result in deleting all the employees who work on that project.
  - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Figure 14.3 Two relation schemas suffering from update anomalies

Figure 14.3
Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.

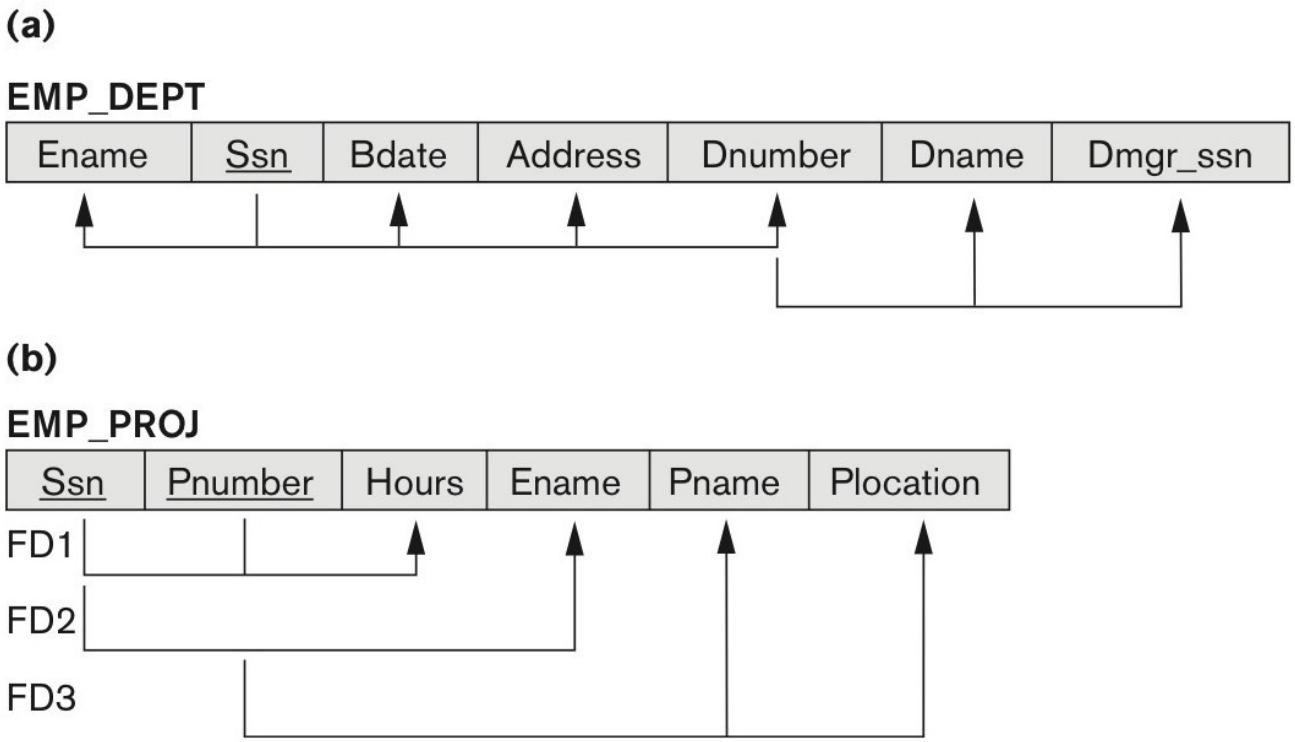


Figure 14.4 Sample states for EMP_DEPT and EMP_PROJ

Figure 14.4
Sample states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 14.2. These may be stored as base relations for performance reasons.

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Guideline for Redundant Information in Tuples and Update Anomalies

- GUIDELINE 2:
  - Design a schema that does not suffer from the insertion, deletion and update anomalies.
  - If there are any anomalies present, then note them so that applications can be made to take them into account.

1.3 Null Values in Tuples

- GUIDELINE 3:
  - Relations should be designed such that their tuples will have as few NULL values as possible
  - Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Reasons for nulls:
  - Attribute not applicable or invalid
  - Attribute value unknown (may exist)
  - Value known to exist, but unavailable

1.4 Generation of Spurious Tuples – avoid at any cost

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations

- GUIDELINE 4:
  - The relations should be designed to satisfy the lossless join condition.
  - No spurious tuples should be generated by doing a natural-join of any relations.

Spurious Tuples (2)

- There are two important properties of decompositions:
  a) Non-additive or losslessness of the corresponding join
  b) Preservation of the functional dependencies.

- Note that:
  - Property (a) is extremely important and *cannot* be sacrificed.
  - Property (b) is less stringent and may be sacrificed. (See Chapter 15).

FUNCTIONAL DEPENDENCIES

2. Functional Dependencies

- Functional dependencies (FDs)
  - Are used to specify *formal measures* of the "goodness" of relational designs
  - And keys are used to define **normal forms** for relations
  - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

2.1 Defining Functional Dependencies

- X ■ Y holds if whenever two tuples have the same value for X, they *must have* the same value for Y
  - For any two tuples t1 and t2 in any relation instance r(R): If t1[X]=t2[X], *then* t1[Y]=t2[Y]
- X ■ Y in R specifies a *constraint* on all relation instances r(R)
- Written as X ■ Y; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes

Examples of FD constraints (1)

- Social security number determines employee name
  - SSN → ENAME
- Project number determines project name and location
  - PNUMBER → {PNAME, PLOCATION}
- Employee ssn and project number determines the hours per week that the employee works on the project
  - {SSN, PNUMBER} → HOURS

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance r(R)
- If K is a key of R, then K functionally determines all attributes in R
  - (since we never have two distinct tuples with t1[K]=t2[K])

Defining FDs from instances

- Note that in order to define the FDs, we need to understand the meaning of the attributes involved and the relationship between them.
- An FD is a property of the attributes in the schema R
- Given the instance (population) of a relation, all we can conclude is that an FD *may exist* between certain attributes.
- What we can definitely conclude is – that certain FDs *do not exist* because there are tuples that show a violation of those dependencies.

Figure 14.7 Ruling Out FDs

Note that given the state of the TEACH relation, we can say that the FD: Text → Course may exist. However, the FDs Teacher → Course, Teacher → Text and Couse → Text are ruled out.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

Figure 14.8 What FDs may exist?

- A relation *R*(A, B, C, D) with its extension.
- Which FDs *may exist* in this relation?

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Figure 14.8 What FDs may exist?

- A relation *R*(A, B, C, D) with its extension.
- Which FDs *may exist* in this relation?

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

$B \rightarrow C$
$C \rightarrow B$
$\{A, B\} \rightarrow C$
$\{A, B\} \rightarrow D$
$\{C, D\} \rightarrow B$

Figure 14.8 What FDs may exist?

- A relation *R*(A, B, C, D) with its extension.
- Which FDs *may exist* in this relation?

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

$B \rightarrow C$
$C \rightarrow B$
$\{A, B\} \rightarrow C$
$\{A, B\} \rightarrow D$
$\{C, D\} \rightarrow B$

B → A (tuples 2 and 3 violate this constraint)
D → C (tuples 3 and 4 violate it)

NORMAL FORMS BASED ON PRIMARY KEYS

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

3.1 Normalization of Relations (1)

- **Normalization:**
  - The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations to **minimize** redundancy and dependency of data

- **Normal form:**
  - Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Normalization of Relations (2)

- 2NF, 3NF, BCNF
  - based on keys and FDs of a relation schema
- 4NF
  - based on keys, multi-valued dependencies : MVDs;
- 5NF
  - based on keys, join dependencies : JDs

3.2 Practical Use of Normal Forms

- **Normalization**
  - is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of higher normal forms (4NF and 5NF) becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
  - (usually up to 3NF and BCNF. 4NF rarely used in practice.)
- **Denormalization**:
  - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

3.3 Definitions of Keys and Attributes Participating in Keys (1)

- A **superkey** of a relation schema R = {A1, A2, ..., An} is a set of attributes S *subset-of* R with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]

- A **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.

Definitions of Keys and Attributes Participating in Keys (2)

- If a relation schema has more than one key, each is called a **candidate** key.
  - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

3.4 First Normal Form

- Disallows
  - composite attributes
  - multivalued attributes
  - **Nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of a relation
- Most RDBMSs allow only those relations to be defined that are in First Normal Form

3.4 First Normal Form

- There are three main techniques to achieve 1NF:
  - Remove the attribute that violates 1NF and place it in a separate relation along with the PK of the main relation
  - Expand the key so that there will be a separate tuple in the original relation for each value
  - If a maximum number of values is know for the attribute, replace the attribute by N atomic attributes

Figure 14.9 Normalization into 1NF

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 14.9
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Sample state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Figure 14.10 Normalizing nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

$\{ssn, Pno\} \subset Hours$

Figure 14.10

Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Sample extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

Ex.

Student_ID	**Student_name**	**Subject**
101	Ahmad	OS, CN
102	Mohammed	Java
103	Ali	C, C++

Ex.

Student_ID	**Student_name**
101	Ahmad
102	Mohammed
103	Ali

Student_ID	**Subject**
101	OS
101	CN
102	Jave
103	C
103	C++

3.5 Second Normal Form (1)

- Uses the concepts of **FDs, primary key**
- Definitions
  - **Prime attribute:** An attribute that is member of the primary key K
  - **Full functional dependency:** a FD Y -> Z where removal of any attribute from Y means the FD does not hold any more
- Examples:
  - {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
  - {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency) since SSN -> ENAME also holds

Second Normal Form (2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization or “second normalization”
- To remove Partial dependency, we can divide the table, remove the attribute which is causing partial dependency, and move it to some other table where it fits in well

Figure 14.11 Normalizing into 2NF

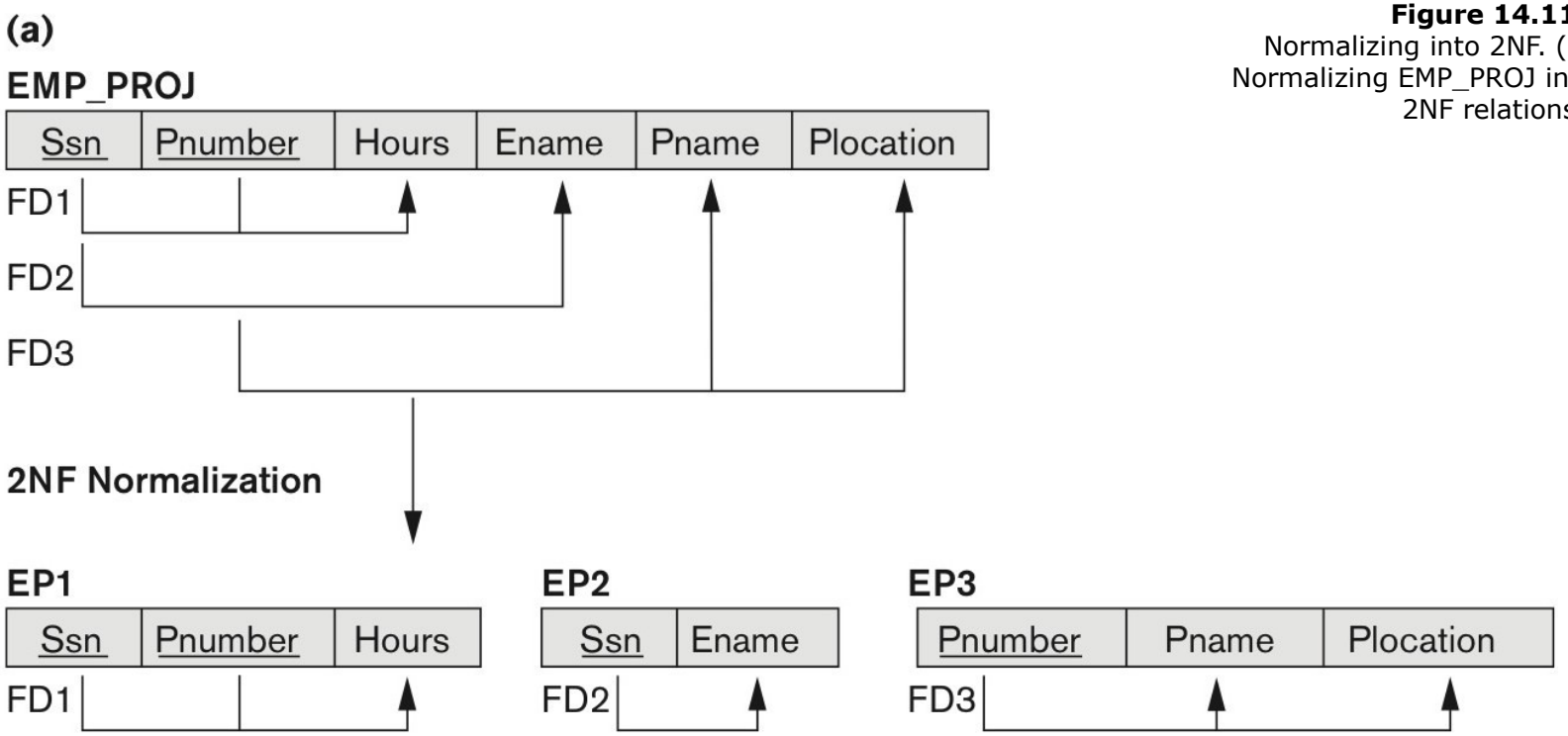


Figure 14.11
Normalizing into 2NF. (a) Normalizing EMP_PROJ into 2NF relations..

Ex.

Score_id	**Student_id**	**Subject_id**	**Mark**	**Teacher**
1	101	1	70	Java teacher
2	101	2	75	C++ teacher
3	102	1	80	Java teacher

{Score_id}
{Student_id, Subject_id}

Ex.

Score_id	**Student_id**	**Subject_id**	**Mark**
1	101	1	70
2	101	2	75
3	102	1	80

Subject_id	**Teacher**
1	Java teacher
2	C++ teacher

3.6 Third Normal Form (1)

- Definition:
  - **Transitive functional dependency:** a FD X -> Z that can be derived from two FDs X -> Y and Y -> Z
  - When a non-prime attribute depends on other non-prime attributes rather than depending upon the prime attributes or primary key
- Examples:
  - SSN -> DMGRSSN is a **transitive** FD
    - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
  - SSN -> ENAME is **non-transitive**
    - Since there is no set of attributes X where SSN -> X and X -> ENAME

Third Normal Form (2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
  - In X -> Y and Y -> Z, with X as the primary key, we consider this a problem **only if** Y is not a candidate key.
  - When Y is a candidate key, there is no problem with the transitive dependency .
  - E.g., Consider EMP (SSN, Emp#, Salary).
    - Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

Figure 14.11 Normalizing into 2NF and 3NF

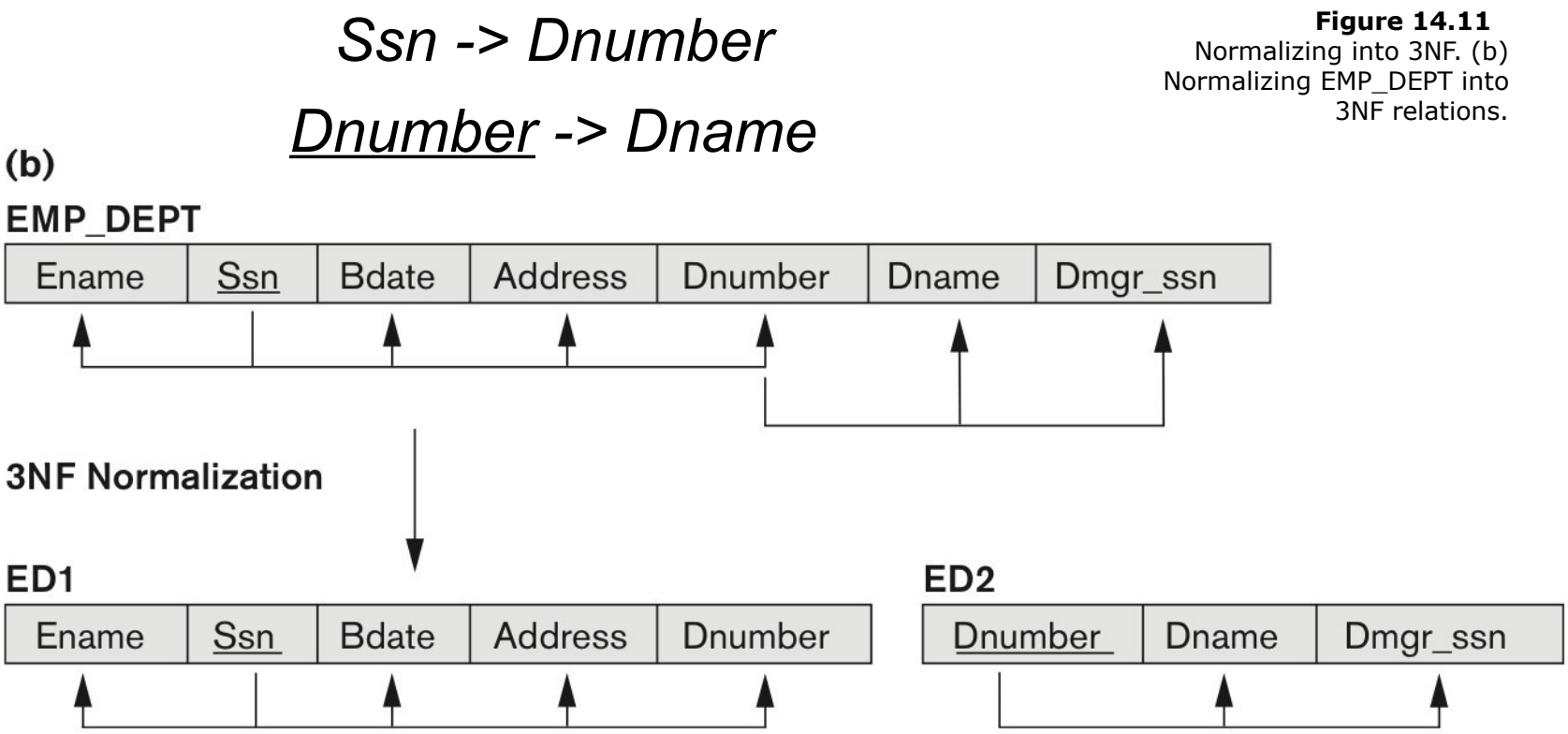


Figure 14.11
Normalizing into 3NF. (b) Normalizing EMP_DEPT into 3NF relations.

Ex.

Score_id	Student_id	Subject_id	Mark	Exam_name	Total_marks
1	101	1	30	Math-midterm	40
2	101	2	25	English	30
3	102	1	33	Math-midterm	40

Exam_name->Total_marks

Non_prime -> non_prime

Ex

Score_id	Student_id	Subject_id	Mark	Exam_name
1	101	1	30	Java teacher
2	101	2	25	C++ teacher
3	102	1	33	Java teacher

Exam_name	Total_marks
C++ teacher	30
Java teacher	40

Normal Forms Defined Informally

- 1st normal form
  - All attributes depend on **the key**
- 2nd normal form
  - All attributes depend on **the whole key** (no partial dependency)
- 3rd normal form
  - All attributes depend on **nothing but the key** (no transitive dependency)

GENERAL DEFINITIONS OF SECOND AND THIRD NORMAL FORMS

4. General Normal Form Definitions (For Multiple Keys) (1)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- Any attribute involved in a candidate key is a *prime attribute*
- All other attributes are called *non-prime attributes.*

4.1 General Definition of 2NF (For Multiple Candidate Keys)

- A relation schema R is in **second normal form (2NF)** if:
  - every non-prime attribute A in R is **fully** functionally dependent on *every* key of R
- In Figure 14.12 the FD

  County_name → Tax_rate violates 2NF.

- So second normalization converts LOTS into

  LOTS1 (Property_id#, County_name, Lot#, Area, Price)

  LOTS2 (County_name, Tax_rate)

Figure 14.12 Normalization into 2NF

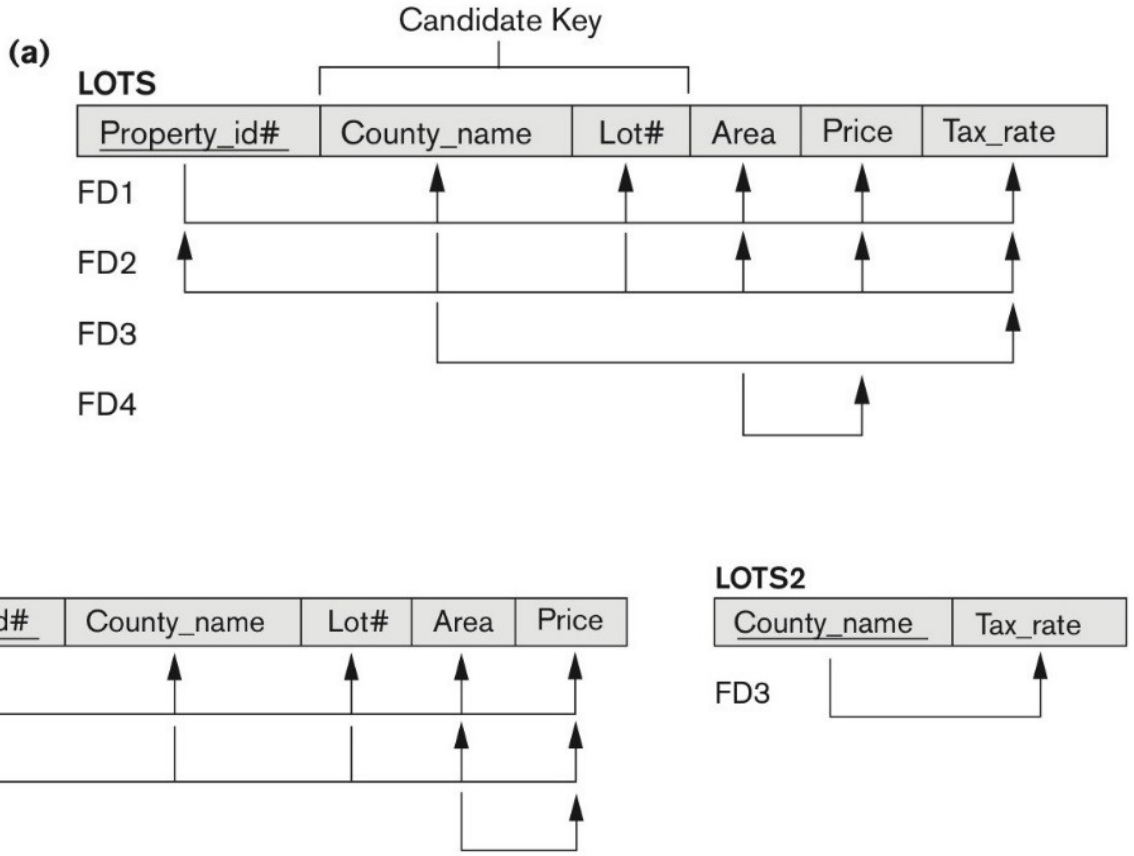


Figure 14.12
Normalization into 2NF.
(a) The LOTS relation with its functional dependencies FD1 through FD4.
(b) Decomposing into the 2NF relations LOTS1 and LOTS2.

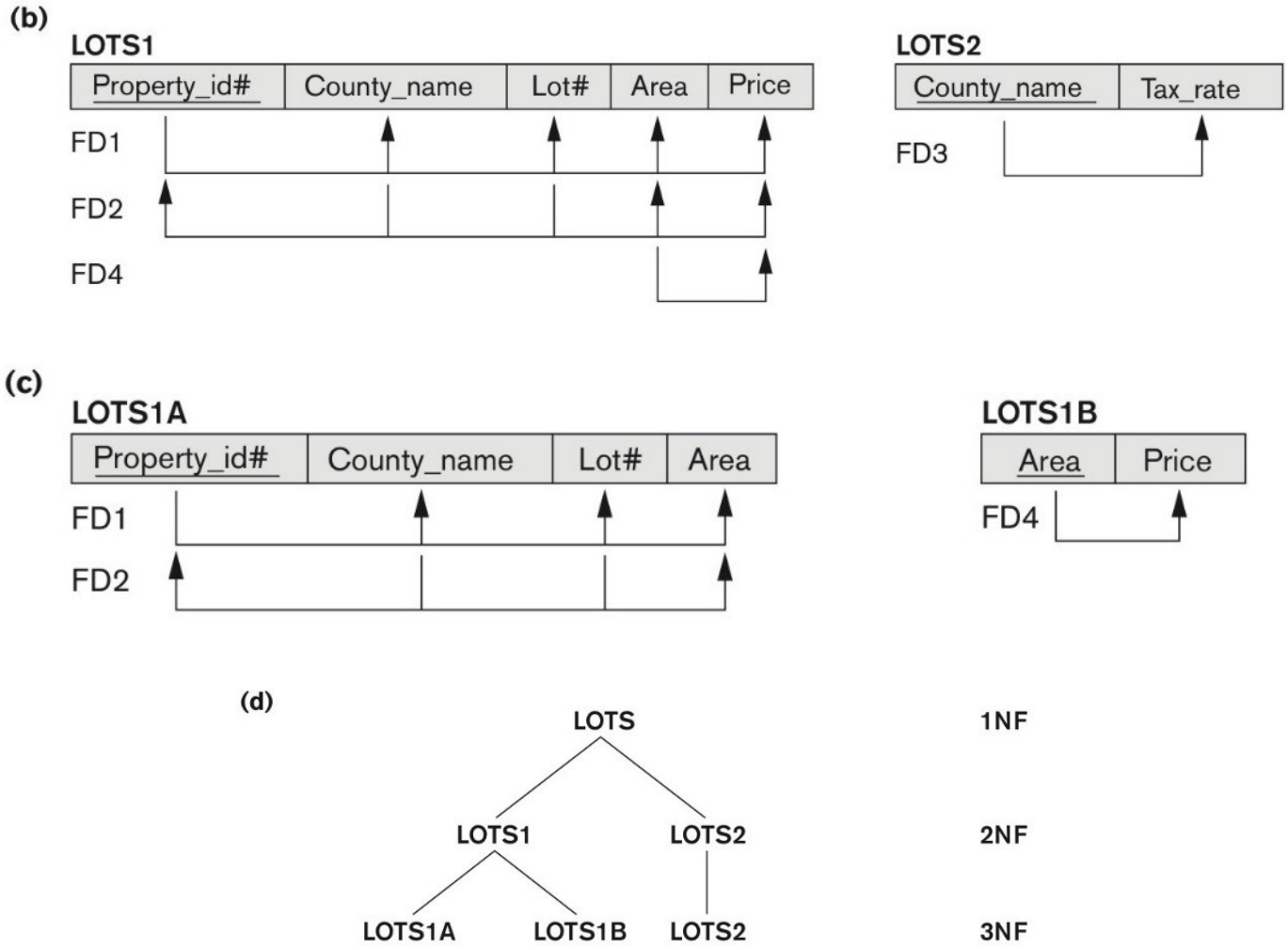
4.2 General Definition of Third Normal Form

- Definition:
  - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
  - A relation schema R is in **third normal form (3NF)** if whenever a FD X → A holds in R, then either:
    - (a) X is a superkey of R, or
    - (b) A is a prime attribute of R
- LOTS1 relation violates 3NF because

  Area → Price ; and Area is not a superkey in LOTS1.

Figure 14.12 Normalization into 2NF and 3NF

Figure 14.12
Normalization into 3NF. (c) Decomposing LOTS into the 3NF relations LOTS1A and LOTS1B. (d) Progressive normalization of LOTS into a 3NF design.



4.3 Interpreting the General Definition of Third Normal Form

- Consider the 2 conditions in the Definition of 3NF:

  A relation schema R is in **third normal form (3NF)** if whenever a FD X → A holds in R, then either:

  - (a) X is a superkey of R, or
  - (b) A is a prime attribute of R

- Condition (a) catches two types of violations :
  - A nonprime attribute determines another nonprime attribute. Here we typically have a transitive dependency that violates 3NF.
  - A proper subset of a key of R functionally determines a nonprime attribute. Here we have a partial dependency that violates 2NF.

4.3 Interpreting the General Definition of Third Normal Form (2)

- **ALTERNATIVE DEFINITION of 3NF: We can restate the definition as:**

  A relation schema R is in **third normal form (3NF)** if every non-prime attribute in R meets both of these conditions:

  - It is fully functionally dependent on every key of R
  - It is non-transitively dependent on every key of R

  Note that stated this way, a relation in 3NF also meets the requirements for 2NF.

- The condition (b) from the last slide takes care of the dependencies that "slip through" (are allowable to) 3NF but are "caught by" BCNF which we discuss next.

BOYCE-CODD NORMAL FORM

5. BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD X → A** holds in R, then **X is a superkey** of R
- Each normal form is strictly stronger than the previous one
  - Every 2NF relation is in 1NF
  - Every 3NF relation is in 2NF
  - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- Hence BCNF is considered a stronger form of 3NF
- The goal is to have each relation in BCNF (or 3NF)

Figure 14.13 Boyce-Codd normal form

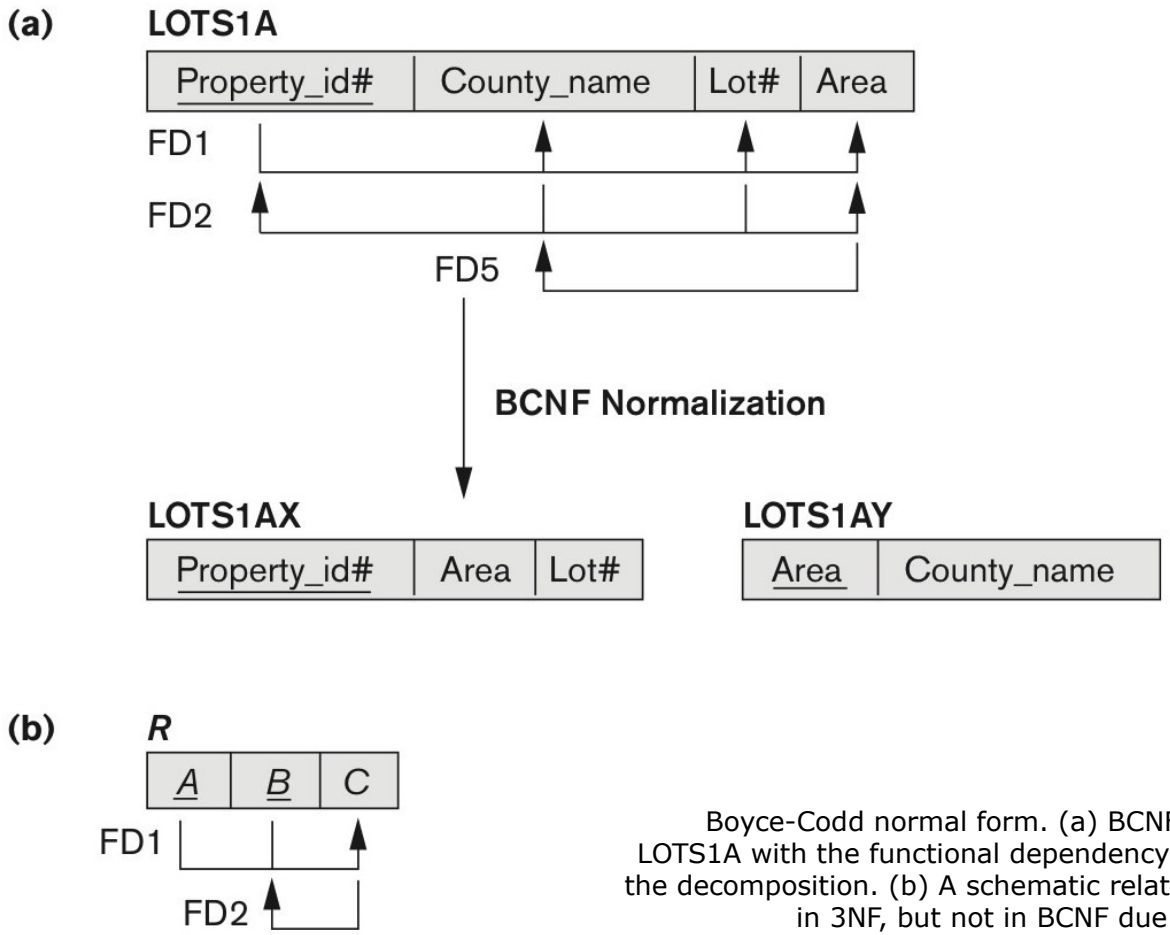
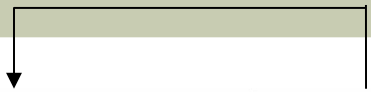


Figure 14.13
Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF due to the f.d. C → B.

A relation TEACH that is in 3NF but not in BCNF



student_id	subject	professor
101	Java	P.Java
101	C++	P.Cpp
102	Java	P.Java2
103	C#	P.Chash
104	Java	P.Java

- One student can enroll for multiple subjects
- For each subject, a professor is assigned to the student
- Multiple professors teaching one subject
- One professor teaches only one subject, but one subject may have two different professors

Achieving the BCNF by Decomposition

- Two FDs exist in the relation TEACH:
  - fd1: { student, subject} -> instructor
  - fd2: instructor -> subject
- {student, subject} is a candidate key for this relation and that the dependencies are OK.
  - So this relation is in 3NF *but not in* BCNF
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

A RELATION TEACH THAT IS IN BCNF

Student Table

student_id	p_id
101	1
101	2
and so on...	

And, **Professor Table**

p_id	professor	subject
1	P.Java	Java
2	P.Cpp	C++
and so on...		

Summary

- 1NF
  - Prime attributes ☾ all other attributes
- 2NF
  - Partial dependency
  - Part of prime attribute ☾ non-prime attribute
- 3NF
  - Transitive dependency
  - non-prime attribute ☾ non-prime attribute
- BCNF
  - Non-prime attribute ☾ prime attribute

MULTIVALUED DEPENDENCY AND FOURTH NORMAL FORM

Figure 14.15 Fourth and fifth normal forms.

(a) EMP

Ename	Pname	Dname
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(b) EMP_PROJECTS

Ename	Pname
Smith	X
Smith	Y

EMP_DEPENDENTS

Ename	Dname
Smith	John
Smith	Anna

Figure 14.15
Fourth and fifth normal forms. (a) The EMP relation with two MVDs: Ename –>> Pname and Ename –>> Dname. (b) Decomposing the EMP relation into two 4NF relations EMP_PROJECTS and EMP_DEPENDENTS.

Multivalued Dependencies and Fourth Normal Form

- **Formal discussion**:
  - Multivalued dependencies are a consequence of 1NF, which disallows an attribute in a tuple to have a set of value
  - If more than one multivalued attribute is present, spreading the values into tuples introduces multivalued dependency
- **Informal discussion**:
  - Whenever two independent 1:N relationships A:B and A:C are mixed in the same relation, R(A, B, C), an MVD may arise

5. Multivalued Dependencies and Fourth Normal Form (1)

Definition:

- A **multivalued dependency** (**MVD**) X —>> Y specified on relation schema R, where X and Y are both subsets of R, specifies the following constraint on any relation state r of R: If two tuples t_1 and t_2 exist in r such that $t_1[X] = t_2[X]$, then two tuples t_3 and t_4 should also exist in r with the following properties, where we use Z to denote (R 2 (X ∪ Y)):
  - $t_3[X] = t_4[X] = t_1[X] = t_2[X]$.
  - $t_3[Y] = t_1[Y]$ and $t_4[Y] = t_2[Y]$.
  - $t_3[Z] = t_2[Z]$ and $t_4[Z] = t_1[Z]$.
- An MVD X —>> Y in R is called a **trivial MVD** if (a) Y is a subset of X, or (b) X ∪ Y = R.

Multivalued Dependencies and Fourth Normal Form

Definition:

- A relation schema *R* is in **4NF** with respect to a set of dependencies *F* (that includes functional dependencies and multivalued dependencies) if, for every *nontrivial* multivalued dependency $X \twoheadrightarrow Y$ in F^+, *X* is a superkey for R.
  - Note: F^+ is the (complete) set of all dependencies (functional or multivalued) that will hold in every relation state *r* of *R* that satisfies *F*. It is also called the **closure** of *F*.

Chapter Summary

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
- Normal Forms (1NF, 2NF, 3NF)Based on Primary Keys
- General Normal Form Definitions of 2NF and 3NF (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)
- Fourth Normal Forms

EXERCISES

Normalization Exercise 1

HEALTH HISTORY REPORT

PET ID	PET NAME	PET TYPE	PET AGE	OWNER	VISIT DATE	PROCEDURE
246	ROVER	DOG	12	SAM COOK	JAN 13/2002	01 - RABIES VACCINATION
					MAR 27/2002	10 - EXAMINE and TREAT WOUND
					APR 02/2002	05 - HEART WORM TEST
298	SPOT	DOG	2	TERRY KIM	JAN 21/2002	08 - TETANUS VACCINATION
					MAR 10/2002	05 - HEART WORM TEST
341	MORRIS	CAT	4	SAM COOK	JAN 23/2001	01 - RABIES VACCINATION
					JAN 13/2002	01 - RABIES VACCINATION
519	TWEEDY	BIRD	2	TERRY KIM	APR 30/2002	20 - ANNUAL CHECK UP
					APR 30/2002	12 - EYE WASH

Normalization Exercise 1 - Solution

- Unormalized form (UNF):
  - Pet [pet_id, pet_name, pet_type, pet_age, owner, (visitdate, procedure_no, procedure_name)]

Normalization Exercise 1 - Solution

- Unormalized form (UNF):
  - Pet [pet_id, pet_name, pet_type, pet_age, owner, (visitdate, procedure_no, procedure_name)]
- 1NF:
  - Pet [pet_id, pet_name, pet_type, pet_age, owner]
  - Pet_Visit [pet_id, visitdate, procedure_no, procedure_name]

Normalization Exercise 1 - Solution

- Unormalized form (UNF):
  - Pet [pet_id, pet_name, pet_type, pet_age, owner, (visitdate, procedure_no, procedure_name)]
- 1NF:
  - Pet [pet_id, pet_name, pet_type, pet_age, owner]
  - Pet_Visit [pet_id, visitdate, procedure_no, procedure_name]
- 2NF:
  - Pet [pet_id, pet_name, pet_type, pet_age, owner]
  - Pet_Visit [pet_id, visitdate, procedure_no]
  - Procedure [procedure_no, procedure_name]

Normalization Exercise 1 - Solution

- Unormalized form (UNF):
  - Pet [pet_id, pet_name, pet_type, pet_age, owner, (visitdate, procedure_no, procedure_name)]
- 1NF:
  - Pet [pet_id, pet_name, pet_type, pet_age, owner]
  - Pet_Visit [pet_id, visitdate, procedure_no, procedure_name]
- 2NF:
  - Pet [pet_id, pet_name, pet_type, pet_age, owner]
  - Pet_Visit [pet_id, visitdate, procedure_no]
  - Procedure [procedure_no, procedure_name]
- 3NF:
  - same as 2NF

Normalization Exercise 2

INVOICE

HILLTOP ANIMAL HOSPITAL
INVOICE # 987

DATE: JAN 13/2002

MR. RICHARD COOK
123 THIS STREET
MY CITY, ONTARIO
Z5Z 6G6

PET	PROCEDURE	AMOUNT
ROVER	RABIES VACCINATION	30.00
MORRIS	RABIES VACCINATION	24.00
	TOTAL	54.00
	TAX (8%)	4.32
	AMOUNT OWING	58.32

Normalization Exercise 2 - Solution

- Unormalized form (UNF):
  - invoice [invoice_no, invoice_date, cust_name, cust_addr, (pet_name, procedure, amount)]

Normalization Exercise 2 - Solution

- Unormalized form (UNF):
  - invoice [invoice_no, invoice_date, cust_name, cust_addr, (pet_name, procedure, amount)]
- 1NF:
  - invoice [invoice_no, invoice_date, cust_name, cust_addr]
  - invoice_pet [invoice_no, pet_id, pet_name, procedure, amount]

Normalization Exercise 2 - Solution

- Unormalized form (UNF):
  - invoice [invoice_no, invoice_date, cust_name, cust_addr, (pet_name, procedure, amount)]
- 1NF:
  - invoice [invoice_no, invoice_date, cust_name, cust_addr]
  - invoice_pet [invoice_no, pet_id, pet_name, procedure, amount]
- 2NF:
  - invoice [invoice_no, invoice_date, cust_name, cust_addr]
  - invoice_pet [invoice_no, pet_id, procedure, amount]
  - pet [pet_id, pet_name]

Normalization Exercise 2 - Solution

- Unormalized form (UNF):
  - invoice [invoice_no, invoice_date, cust_name, cust_addr, (pet_name, procedure, amount)]
- 1NF:
  - invoice [invoice_no, invoice_date, cust_name, cust_addr]
  - invoice_pet [invoice_no, pet_id, pet_name, procedure, amount]
- 2NF:
  - invoice [invoice_no, invoice_date, cust_name, cust_addr]
  - invoice_pet [invoice_no, pet_id, procedure, amount]
  - pet [pet_id, pet_name]
- 3NF:
  - invoice [invoice_no, invoice_date, cust_no (FK)]
  - invoice_pet [invoice_no (FK), pet_id (FK), procedure, amount]
  - pet [pet_id, pet_name]
  - customer [cust_no, cust_name, cust_street, cust_city]

Normalization Exercise 3

Gallery Customer History Form

Customer Name

Jackson, Elizabeth
123 – 4th Avenue
Fonthill, ON
L3J 4S4

Phone (206) 284-6783

Purchases Made

Artist	Title	Purchase Date	Sales Price
03 - Carol Channing	Laugh with Teeth	09/17/2000	7000.00
15 - Dennis Frings	South toward Emerald Sea	05/11/2000	1800.00
03 - Carol Channing	At the Movies	02/14/2002	5550.00
15 - Dennis Frings	South toward Emerald Sea	07/15/2003	2200.00

The Gill Art Gallery wishes to maintain data on their customers, artists and paintings. They may have several paintings by each artist in the gallery at one time. Paintings may be bought and sold several times. In other words, the gallery may sell a painting, then buy it back at a later date and sell it to another customer.

Normalization Exercise 3 - Solution

- Unormalized form (UNF):
  - customer [custno, cust_name, cust_addr, cust_phone, (artist_id, artist_name, art_title, pur_date, price)]
- 1NF:
  - customer [custno, cust_name, cust_addr, cust_phone]
  - cust_art [custno, art_code, pur_date, artist_id, artist_name, art_title, price]
- 2NF:
  - customer [custno, cust_name, cust_addr, cust_phone]
  - cust_art [custno, art_code, pur_date, price]
  - art [art_code, art_title, artist_id, artist_name]
- 3NF:
  - customer [custno, cust_name, cust_street, cust_city, cust_phone]
  - cust_art [custno, art_code, pur_date, price]
  - art [art_code, art_title, artist_id(FK)]
  - artist [artist_id, artist_fname, artist_lname]